

Introduction to Next.js

Build
Learn
Grow
Repeat... 😊

"The React Framework for the Web"

1 What is Next.js?

Next.js is a React framework developed by **Vercel** that helps us build fast, scalable and production-ready web applications. It provides a lot of features out of the box like routing, server-side rendering, static site generation, API routes and more.

NEXT.JS

Maintained by : **Vercel** ▲
First release : 2016
Built on : React
Language : JavaScript/TypeScript

2 React vs Next.js

React (Library)	Next.js (Framework)
UI library	Full-fledged framework
Client-side rendering (by default)	Supports SSR, SSG, ISR etc.
Routing (using React Router)	Built-in file based routing
No API routes	Built-in API routes
Manual configuration (webpack, babel etc.)	Zero configuration (most cases)
Only front-end	Full-stack capabilities
Used for SPAs	Used for real world web apps

"Next.js makes React more powerful and production ready."

💡 In short:

React helps you build UI but Next.js helps you build complete web applications with better performance, SEO and developer experience.

Next.js = React
+
Superpowers ⚡

3 Why Next.js?

- ✓ Better performance (SSR, SSG, ISR)
- ✓ SEO friendly
- ✓ File-based routing (easy to use)
- ✓ Full-stack capabilities (API routes)
- ✓ Automatic code splitting
- ✓ Optimized image and font handling
- ✓ Large community and great documentation
- ✓ Developed and maintained by Vercel (used in production)

★ Popular Apps using Next.js

- Vercel (of course!)
- Netflix (marketing site)
- TikTok (marketing site)
- Notion
- Twitch
- GitHub (some parts)
- Discord (marketing site)
- and many more ...

4 Where is Next.js used?

- Static websites (blogs, docs, marketing sites)
- Dynamic web applications (dashboards, SaaS products)
- E-commerce platforms
- Company websites and landing pages
- Anything that needs performance + SEO + scalability

You can build almost any kind of web application with Next.js!

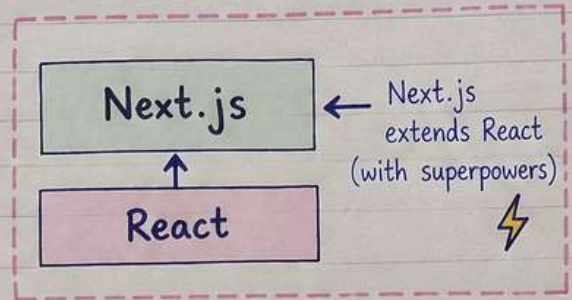
😊

Next.js Architecture

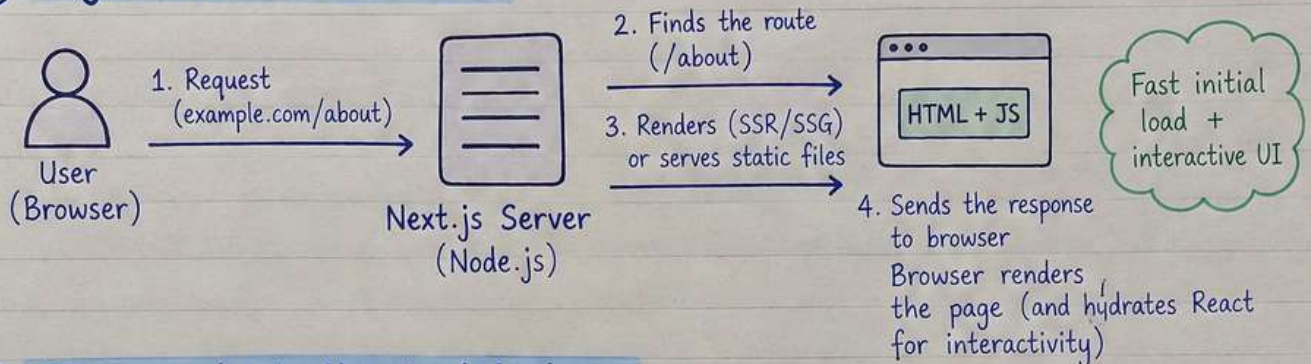
Same React
More Power
with Next.js! 😊

1 Big Picture

Next.js is built on top of React. It extends React with additional features like server-side rendering, static site generation, file-based routing, API routes and more, making it easier to build full-stack web applications.



2 High Level Architecture



3 Components in the Architecture

Part	Role
React	UI library (for building components)
Next.js	Framework (routing, rendering, API routes, optimizations, etc.)
Node.js	Runtime environment (runs Next.js server)
Browser	Renders the UI and makes it interactive (hydration)
Vercel (optional)	Platform to deploy and host Next.js apps (optimized for Next.js)

Key Takeaways:

- Next.js = React + additional features.
- Can render pages on server (SSR) or at build time (SSG).
- Uses file-based routing instead of manual route config.
- Supports both frontend and backend (API routes).
- Optimized for production (automatic code splitting, image optimization, caching).

4 Request → Response Flow (Example)

1. User visits `https://example.com/about`
2. Next.js server checks for `/app/about/page.js` (or `/pages/about.js`)
3. If using SSR → server renders the page at request time.
If using SSG → serves pre-built static HTML.
4. HTML (and JS) is sent to the browser.
5. Browser displays the page and loads React for interactivity (hydration).

"Same React components ... but more ways to render!" 😊

5 Mental Model

Think of Next.js as a toolbox on top of React that helps you build real-world web applications faster, with better performance and developer experience.

React = Engine
Next.js = Complete Car
(ready for production)



Creating a Next.js App

First Project
Let's Go!! 😊

1 Prerequisites

- Node.js (v18.17 or later)
- npm (comes with Node.js) or yarn / pnpm
- A code editor (VS Code recommended)
- Basic knowledge of JavaScript and React

💡 Check Node & npm version

```
node -v    # e.g. v20.11.0
npm -v     # e.g. 10.2.4
```

2 Create a New Next.js App

- Using create-next-app (recommended)

```
npx create-next-app@latest my-next-app
```

→ Creates a new Next.js project with the latest features and best practices.

3 During Setup (You'll see some prompts)

```
? Would you like to use TypeScript? No / Yes
? Would you like to use ESLint? No / Yes
? Would you like to use Tailwind CSS? No / Yes
? Would you like to use 'src/' directory? No / Yes
? Would you like to use App Router? (Yes)
? Would you like to customize the import alias? No / Yes
```

→ You can choose based on your preference.
For learning, you can keep most options as **Yes** 😊

4 Run the Development Server

```
cd my-next-app
npm run dev
```

- Moves into the project folder
- Starts the development server

Now open in your browser: → <http://localhost:3000>

You should see the default Next.js welcome page 🎉

5 Other Useful Commands

Command	Description
npm run build	Builds the app for production
npm run start	Runs the production build
npm run lint	Runs ESLint to check code quality
npm install <package>	Installs a package
npm run dev -- -p 3001	Runs dev server on a different port

By default dev server runs on port 3000.
You can change it using -p flag.

Pro Tip: Use **pnpm** instead of npm (faster and efficient)

You're ready to explore the project structure! 😊

Next.js Project Structure

"Good project structure makes big projects simple" 😊

1 Overview

When you create a Next.js app using 'create-next-app', it already sets up a good folder structure for you.

It follows the **App Router** (in Next.js 13+), which uses the 'app/' directory instead of 'pages/'.

2 Typical Project Structure

my-next-app/



★ Key Folders

app/	Main folder for routes, layouts and pages (App Router).
public/	Stores static assets (images, icons, fonts etc.) Access via / (e.g. /logo.png)
src/	Optional. You can keep your source code here (e.g. app/, components/).
node_modules/	Installed packages (don't touch manually).

Why App Router?

- ✓ More powerful and flexible than the old Pages Router.
- ✓ Supports layouts, nested routes, server components and more.
- ✓ Better for modern React and future features.
- ✓ Still, Pages Router (pages/) is supported for older projects.

3 Important Files

File	Purpose
app/layout.js	Defines the root layout for your app.
app/page.js	Home page (/ route).
next.config.js	Configuration file for Next.js.
package.json	Project info, dependencies and scripts.
public/	Static files (directly accessible).
globals.css	Global CSS (imported in layout.js).
.gitignore	Specifies files/folders to ignore in Git.

4 Useful Scripts (in package.json)

npm run dev	Start development server
npm run build	Build for production
npm run start	Start production server
npm run lint	Run ESLint

5 Quick Tips

- You can use 'src/' folder to keep your code organized (recommended for large apps).
- Keep components in a separate folder (e.g. 'src/components/').
- Use meaningful folder names for routes.
- Static assets go inside 'public/'.
- You don't need to manually configure most things - Next.js handles it for you 😊

Next Page: → Routing in Next.js (App Router)

Structure today
Success tomorrow! 😊

Routing in Next.js (App Router)

"File based routing makes routing super easy!" 😊

1 What is Routing?

Routing means mapping a URL (route) to a particular page or component. In Next.js (App Router), routes are created **automatically** based on the folder structure inside the **'app/'** directory.

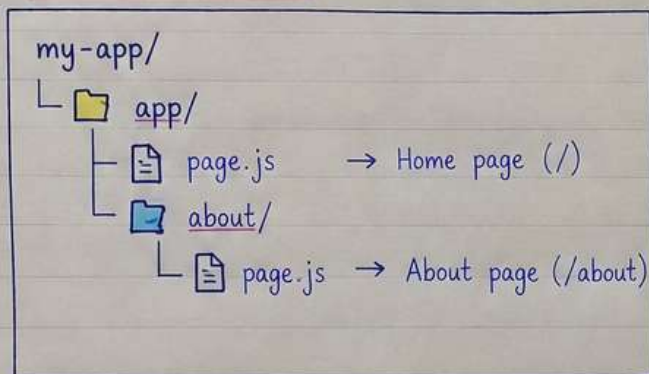
💡 Key Point:

In App Router, we use **'app/'** folder (not **'pages/'**) for defining routes. Each folder becomes a route and each **'page.js'** becomes a page.

2 Basic Routing Example

Let's create a simple Next.js app with two routes: **/** and **/about**

Folder Structure



Example Code

```

app/page.js (Home page)
export default function Home() {
  return <h1>Welcome to Next.js</h1>;
}
  
```

```

app/about/page.js (About page)
export default function About() {
  return <h1>About Us</h1>;
}
  
```

3 How it Works?

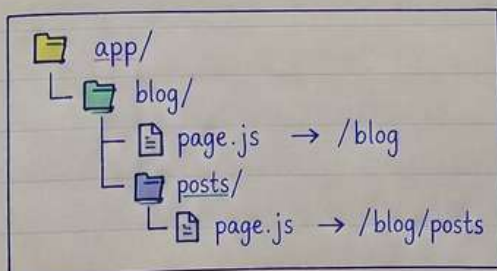
- 1 You create a folder inside **app/** (e.g. **about**)
- 2 Add a **page.js** file inside it
- 3 Next.js **automatically** maps it to the route (e.g. **/about**)
- 4 No manual route configuration needed!

URL → File Mapping

URL	File
/	app/page.js
/about	app/about/page.js
/contact	app/contact/page.js
/blog	app/blog/page.js

4 Nested Routes (Preview)

You can also create nested folders for nested routes.



This makes it easy to organize related pages together.

Why File-based Routing?

- ✓ Simple and intuitive
- ✓ No extra config required
- ✓ Scales well for large apps
- ✓ Supports nested and dynamic routes
- ✓ Works for both static and dynamic rendering

Quick Note:

- In older versions of Next.js, we used **'pages/'** directory for routing.
- In Next.js 13+ (App Router), **'app/'** is the recommended way.

Next.js takes care of routing, so you can focus on building amazing features!! 😊

Dynamic & Nested Routes in Next.js

Dynamic
Data
Dynamic
Routes
More Possibilities!
😊

1 What are Dynamic Routes?

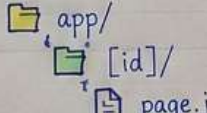
Dynamic routes allow you to create pages that can handle dynamic data by using square brackets `[]` in the folder name.

Useful when you don't know the exact value of the route in advance (e.g. user id, product id, slug etc.)

Syntax:

Folder name: `[param]`

Example:



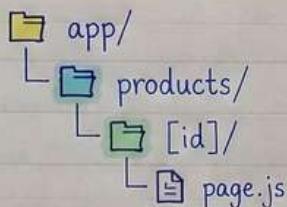
`app/`
`[id]/`
`page.js`

This will match `/1`
`/2` `/3` etc.

2 Example: Dynamic Route

Let's create a product details page: `/products/[id]`

Folder Structure



`app/`
`products/`
`[id]/`
`page.js`

`app/products/[id]/page.js`

```
export default function ProductPage({ params }) {
  return (
    <div>
      <h1>Product ID: {params.id}</h1>
    </div>
  )
}
```

★ Try it!

Visit `/products/101`
It will render the page with `params.id = 101`
😊

3 Catch-all Routes

Catch-all routes allow you to match multiple segments in the URL using `...` before the folder name.

Example: `/docs/a/b/c` → `app/docs/[...slug]/page.js`

`app/docs/[...slug]/page.js`

```
export default function DocsPage({ params }) {
  return <div>Path: {params.slug.join(' / ')}</div>
}
```



When to use?

- Documentation sites
- Blog categories
- File systems
- Any nested dynamic content

URL Examples

<code>/docs/intro</code>	→	<code>['intro']</code>
<code>/docs/a/b</code>	→	<code>['a', 'b']</code>
<code>/docs/a/b/c</code>	→	<code>['a', 'b', 'c']</code>

params.slug

4 Optional Catch-all Routes

Use `[[...slug]]` (double brackets) to make the parameter optional.

`app/docs/[[...slug]]/page.js`

```
export default function DocsPage({ params }) {
  return <div>Path: {params.slug ? params.slug.join(' / ') : 'Home'}</div>
}
```

URL Examples

<code>/docs</code>	→	<code>undefined</code>
<code>/docs/intro</code>	→	<code>['intro']</code>
<code>/docs/a/b</code>	→	<code>['a', 'b']</code>

params.slug

5 Nested Routes Recap

You can combine static, dynamic and catch-all routes to create complex route structures easily.

Next.js routing is powerful and flexible — just folders!
😊

Note:

No need to define routes manually (like in React Router). It's file-based
😊

Server vs Client Components in Next.js

Understand this well — it's important 😊

1 What are Server and Client Components?

In Next.js (App Router), components are Server Components by default. We can also create Client Components when we need interactivity (state, event handlers, etc).

Key Idea:

Server Components → run on the server
(Client doesn't get their JS)

Client Components → run in the browser
(Need "use client")

2 Comparison

Feature	Server Component	Client Component
Where it runs	Server (Node.js)	Browser
Default in Next.js	Yes	No (must add "use client")
Can use state (useState)	No	Yes
Can use event handlers (onClick, etc)	No	Yes
Access to server resources (DB, API keys)	Yes	No (exposed to browser)
Bundle size	Not included in client JS	Included in client JS
Use case	Data fetching, static content, heavy components	Interactive UI, forms, user events, browser APIs

When to use which?

- Use Server Components as much as possible.
- Use Client Components only when you need interactivity (state, events, browser APIs).
- This helps in better performance and smaller bundle size.

3 Example — Server Component

app/page.js (Server Component)

```
export default async function Home() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts');
  const posts = await res.json();

  return (
    <div>
      <h1>Latest Posts</h1>
      <ul>
        {posts.slice(0, 5).map(p => (
          <li key={p.id}>{p.title}</li>
        ))}
      </ul>
    </div>
  )
}
```

Runs on server
(no client JS sent)

Can directly fetch data from APIs/DB ✓

4 Example — Client Component

app/counter.js (Client Component)

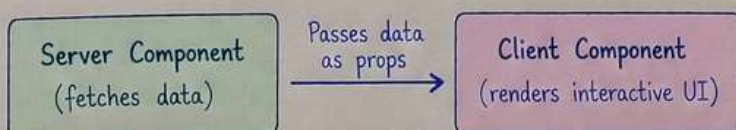
```
"use client" // very important!
import { useState } from 'react'
export default function Counter() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}> ✓
        Increment
      </button>
    </div>
  )
}
```

Runs in browser

Needed for interactivity (state, events)

5 How They Work Together?



Quick Tips:

- Don't use "use client" unless needed.
- Keep Client Components small.
- You can nest Client Components inside Server Components.
- Server Components cannot use browser-only APIs like window, localStorage, etc.

Note: Server Components + Client Components = Best of both worlds! 😊

Less JS, More Performance 😊

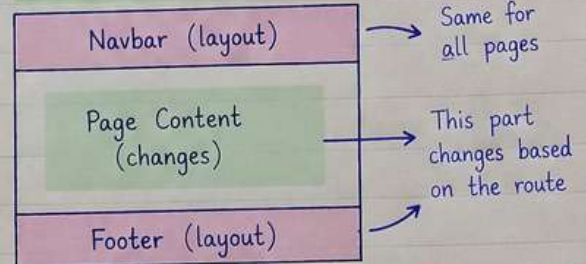
Layouts & Pages in Next.js

"Layouts help you build consistent UI across your app" 😊

1 What are Layouts ?

- A layout is a React component that wraps multiple pages and provides a shared UI (e.g. navbar, footer, sidebar).
- In Next.js (App Router), you can create a layout using 'layout.js' file.
- Layouts **persist** between route changes (only the page content changes).

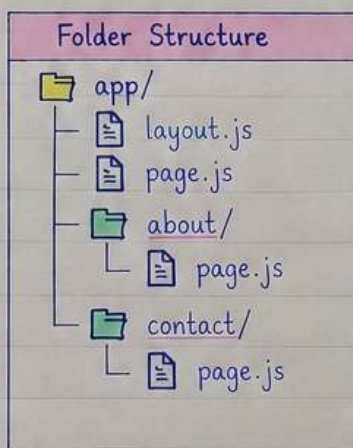
How it looks ?



2 Root Layout Example

In the 'app/' directory, create a 'layout.js' file (root layout).

★ Root layout is required in every Next.js app.



app/layout.js

```
import './globals.css'
export const metadata = {
  title: 'My Next.js App',
  description: 'Learn Next.js'
}
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <nav>My Navbar</nav>
        <main>{children}</main>
        <footer>My Footer</footer>
      </body>
    </html>
  )
}
```

{children} is a special prop that will render the current page inside the layout. 😊

3 Page Example

Each route has a 'page.js' file which is the actual page UI rendered inside the layout.

app/page.js (Home Page)

```
export default function Home() {
  return (
    <div>
      <h1>Welcome to Next.js</h1>
    </div>
  )
}
```

app/about/page.js (About Page)

```
export default function About() {
  return (
    <div>
      <h1>About Us</h1>
    </div>
  )
}
```

Result

/
→ Home page with layout

/about
→ About page with same layout (navbar + footer)

4 Nested Layouts

You can also have nested layouts for specific sections of your app.



Here, dashboard/layout.js will only apply to routes under /dashboard.

Useful for large apps to have different layouts for different sections! 😊

5 Other Useful Files

File	Purpose
layout.js	Defines shared UI for a route (or group of routes)
page.js	The actual page UI
loading.js	UI to show while page is loading (suspense)
error.js	UI for error handling
not-found.js	Shown when route is not found (404)

Pro Tip: Keep your layouts simple and reusable.

Build once, reuse everywhere! 😊

Data Fetching in Next.js

"Good data makes a great UI" 😊

1 Why Data Fetching?

- Most real world apps need data from an external source (database, API, CMS etc).
- Next.js gives multiple ways to fetch data, mainly in Server Components (recommended).
- Helps in better performance, SEO and user experience.

2 Basic Fetch Example (Server Component)

Create a page that fetches posts from a public API.

app/posts/page.js

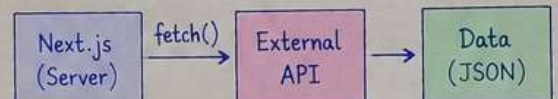
```
export default async function Posts() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts');
  const posts = await res.json();
  return (
    <div>
      <h1>Posts</h1>
      <ul>
        {posts.slice(0, 5).map(post => (
          <li key={post.id}>{post.title}</li>
        ))}
      </ul>
    </div>
  )
}
```

Key Points:

- Fetch data on the server whenever possible.
- Avoid fetching in client components unless necessary.
- Next.js uses `fetch()` with built-in caching and revalidation.
- You can also use third-party libraries like `axios`, `swr`, `react-query`.

How it works?

- 1 `fetch()` runs on the server (during build or request).
- 2 Data is fetched from the API.
- 3 The page is rendered with the data (HTML sent to browser).
- 4 No need to use `useEffect` or `useState`.



3 Using async/await in Server Components

In App Router, components can be `async`, so you can directly use `await` inside the component.

app/users/page.js

```
export default async function Users() {
  const res = await fetch('https://jsonplaceholder.typicode.com/users');
  const users = await res.json();
  return (
    <div>
      <h1>Users</h1>
      <ul>
        {users.map(user => (
          <li key={user.id}>{user.name} - {user.email}</li>
        ))}
      </ul>
    </div>
  )
}
```

4 Fetching in Client Components

If you need to fetch data on the client (e.g. based on user interaction), use `useEffect`.

app/client-data/page.js

```
"use client"
import { useEffect, useState } from 'react'
export default function ClientData() {
  const [data, setData] = useState([])
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(res => res.json())
      .then(setData)
  }, [])
  return <div>{data.length} posts loaded</div>
}
```

5 Things to Remember

- Use Server Components for data fetching whenever possible.
- `fetch()` is cached by default in Next.js (static rendering).
- Use `{ cache: 'no-store' }` for dynamic data (no caching).
- You can also set `revalidate` time for ISR (e.g. `{ next: { revalidate: 60 } }`).
- For client side fetching, use `useEffect` or libraries like `SWR`/React Query.

Quick Example:

```
await fetch('https://api.com/data', {
  cache: 'no-store' // no caching
})
Useful for always fresh data! 😊
```

Next Page: → Rendering in Next.js (SSR, SSG, ISR, CSR)

Data powers applications 😊

API Routes in Next.js

"Build your backend inside your Next.js app" 😊

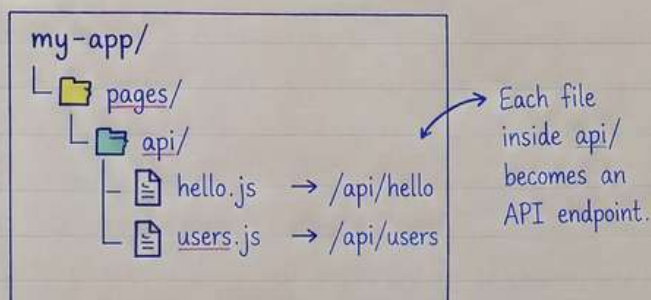
1 What are API Routes ?

- Next.js allows you to create backend APIs inside your project using API Routes.
- These run on the server (Node.js) and handle requests like **GET, POST, PUT, DELETE** etc.
- Useful for small to medium backend logic (e.g. form submissions, auth, database queries).

Key Points:

- API Routes are server-side only (not accessible from browser).
- Located inside `/pages/api/` (or `/app/api/` in newer Next.js).
- Each file becomes an API endpoint.
- No need for a separate backend for simple use cases.

2 Folder Structure



3 Basic API Route Example (GET)

```

pages/api/hello.js
export default function handler(req, res) {
  if (req.method === 'GET') {
    res.status(200).json({ message: 'Hello from Next.js API' });
  } else {
    res.status(405).json({ message: 'Method Not Allowed' });
  }
}
  
```

Visit `/api/hello` in your browser

4 Handling Different Methods

```

pages/api/users.js
export default function handler(req, res) {
  const { method } = req;

  if (method === 'GET') {
    // get all users
    res.status(200).json({ users: ['Abhishek', 'Rohit'] });
  } else if (method === 'POST') {
    // create a user
    const { name } = req.body;
    res.status(201).json({ message: 'User ${name} created' });
  } else {
    res.status(405).json({ message: 'Method Not Allowed' });
  }
}
  
```

5 Accessing API from Client

Using fetch():

```

app/page.js (or any client component)
async function getData() {
  const res = await fetch('/api/hello');
  const data = await res.json();
  console.log(data);
}

getData();
  
```

You can call your own API endpoints from client components using `fetch()` 😊

Response Format:

```

{
  "message": "Hello from Next.js API"
}
  
```

Typically return JSON response

6 Use Cases

- Form submissions
- Authentication (login/signup)
- Database operations
- Proxying third-party APIs
- Handling webhooks (e.g. Stripe, GitHub)
- Any custom backend logic

Pro Tip:

- For larger applications, consider using a dedicated backend (e.g. Node.js + Express, or serverless functions).
- Use environment variables for secrets (API keys, DB URLs).
- Keep your API routes modular and organized.

Quick Test:

- 1 Create `/api/hello` route
- 2 Run your Next.js app
- 3 Open `http://localhost:3000/api/hello`
- 4 You should see the JSON response 😊

Note: In Next.js 13+ (App Router), you can also create API routes using `/app/api/route.js` (more flexible). 😊

Small APIs
Big Possibilities! 😊

State Management in Next.js

"Right state, better experience" 😊

1 Types of State in Next.js

- Local State → `useState` (within a component)
- Shared State → `useContext` or state management libs (across components)
- Server State → fetched from API / database (use `Server Components`, `fetch`, or libraries like `React Query`)
- URL State → route params, query params (use `Router`, `useSearchParams`)
- Persistent State → `localStorage`, cookies, etc.

💡 Key Takeaway:

- Use the right type of state for the right use case.
- Avoid overusing global state.
- Prefer server components for data fetching when possible.
- Keep client state minimal.

2 Local State (`useState`)

Used for component-level state.

app/counter/page.js

```
"use client"
import { useState } from 'react'

export default function Counter() {
  const [count, setCount] = useState(0)
  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  )
}
```

Simple state in a client component

3 Shared State (Context API)

Use Context when you need to share state across multiple components (without prop drilling).

app/context/ThemeContext.js

```
"use client"
import { createContext, useContext, useState } from 'react'

const ThemeContext = createContext()

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light')
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  )
}

export const useTheme = () => useContext(ThemeContext)
```

Provide state to entire app

4 Server State (Data Fetching)

Use `Server Components` (default) to fetch data on the server.

app/products/page.js

```
export default async function Products() {
  const res = await fetch('https://fakestoreapi.com/products')
  const products = await res.json()
  return (
    <div>
      <h1>Products</h1>
      <ul>
        {products.map(p => (
          <li key={p.id}>{p.title}</li>
        ))}
      </ul>
    </div>
  )
}
```

Runs on server (no `useState` needed)

5 URL State (Routing)

Use route params and query params to handle state in URL.

app/search/page.js

```
"use client"
import { useSearchParams } from 'next/navigation'

export default function Search() {
  const searchParams = useSearchParams()
  const query = searchParams.get('q')
  return <div>Search: {query}</div>
}
```

Example URL

/search?q=nextjs

😊

Access in component:

`searchParams.get('q')`

6 Persistent State (`localStorage` / Cookies)

Use for data that needs to persist across sessions.

Using `localStorage` (client only)

```
"use client"
useEffect(() => {
  localStorage.setItem('name', 'Abhishek')
  const name = localStorage.getItem('name')
  console.log(name)
}, [1])
```

Using Cookies (server/client)

```
import { cookies } from 'next/headers'

export default function getName() {
  const cookieStore = cookies()
  const name = cookieStore.get('name')
  return name?.value
}
```

💡 Pro Tips:

- Use server state for initial data.
- Use client state for interactivity.
- Keep global state minimal (use Context or libraries like `Zustand` if needed).
- Use URL state for shareable / bookmarkable state (filters, search, etc).
- Use cookies for auth and user preferences.

Note: Next.js gives flexibility — you can combine multiple state types in a single app! 😊

State smartly,
build better! 😊

Static Generation (SSG) in Next.js

"Pre-render today
for a faster tomorrow"

1 What is Static Generation (SSG)?

- Static Generation means pages are pre-rendered at **build time** and served as static HTML files.
- Best for content that doesn't change often (e.g. blog posts, documentation, marketing pages).
- Very fast, **SEO** friendly and reduces **server** load.



Key Benefits:

- Super fast page loads
- Better **SEO**
- Lower server costs (served via **CDN**)
- Great for static / mostly static content

2 Example - Static Page

A simple page that is statically generated.

pages/about.js

```
export default function About() {
  return (
    <div>
      <h1>About Us </h1>
      <p>This page is statically generated
        at build time.</p>
    </div>
  )
}
```

This page will
be generated as
static HTML during
'next build' 😊

4 When to use SSG?

- Blog / documentation sites
- Marketing or landing pages
- Product pages (with infrequent changes)
- Any content that doesn't change often



Tip:

Use 'getStaticPaths' for dynamic routes (e.g. /blog/[id]) to pre-render multiple pages at build time.

5 SSG with Dynamic Routes

pages/blog/[id].js

```
export async function getStaticPaths() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts')
  const posts = await res.json()

  const paths = posts.slice(0, 5).map(post => ({
    params: { id: post.id.toString() }
  }))

  return { paths, fallback: false } // only these pages
}
```

```
export async function getStaticProps({ params }) {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts/${params.id}')
  const post = await res.json()
  return { props: { post } }
}
```

Pre-renders 5 pages
at build time.
(others will 404)

3 Static Generation with Data

Use 'getStaticProps' to fetch data at build time.

pages/blog.js

```
export async function getStaticProps() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts')
  const posts = await res.json()

  return {
    props: { posts } // passed to the page component
  }
}

export default function Blog({ posts }) {
  return (
    <div>
      <h1>Blog Posts </h1>
      <ul>
        {posts.slice(0, 5).map(post => (
          <li key={post.id}>{post.title}</li>
        ))}
      </ul>
    </div>
  )
}
```

Data is fetched
at build time
and HTML is
generated with
the data 😊

6 SSG vs SSR vs CSR

Feature	SSG	SSR	CSR
When it runs	Build time	On each request	In browser
Data freshness	Static (until rebuild)	Always fresh	Always fresh
Performance	Very fast	Slower (server rendering)	Depends (on network)
SEO	Excellent	Good	Medium
Best for	Static content	Dynamic content	Highly interactive UI



Quick Notes:

- Rebuild your app when content changes.
- You can use **Incremental Static Regeneration (ISR)** for time-based updates (see next page).
- SSG works great with a **CDN** (e.g. **Vercel**).
- Keep your build size in check for large sites.

Fast pages. Happy users!

Advanced Concepts in Next.js

"Go beyond basics,
That's where the real
power is" 😊

1 Middleware in Next.js

- Middleware runs before a request is completed.
- Useful for authentication, redirects, logging, A/B testing, geo-based routing etc.
- It runs on the edge (before reaching your route).

middleware.js (in root)

```
import { NextResponse } from 'next/server'

export function middleware(req) {
  const { pathname } = req.nextUrl
  // Redirect if not logged in
  if (pathname.startsWith('/dashboard')) {
    const token = req.cookies.get('token')
    if (!token) {
      return NextResponse.redirect(new URL('/login', req.url))
    }
  }
  return NextResponse.next()
}
```

Middleware is powerful but keep it lightweight !

2 API Routes (Advanced)

- You can handle dynamic routes, multiple methods and even connect to databases.
- Useful for building full-stack applications.

pages/api/user/[id].js

```
export default async function handler(req, res) {
  const { id } = req.query
  if (req.method === 'GET') {
    // fetch user from DB
    const user = { id, name: 'Abhishek' }
    return res.status(200).json(user)
  } else if (req.method === 'DELETE') {
    // delete user from DB
    return res.status(200).json({ message: 'User deleted' })
  } else {
    res.status(405).json({ message: 'Method Not Allowed' })
  }
}
```

API Routes make Next.js a full-stack framework 😊

3 Image Optimization (next/image)

- Next.js provides an optimized `<Image />` component for better performance.
- It automatically handles resizing, lazy loading and modern formats (WebP).

import Image from 'next/image'

```
export default function Profile() {
  return (
    <Image
      src="/profile.jpg"
      alt="Profile"
      width={200}
      height={200}
      priority // loads faster (use for LCP images)
    />
  )
}
```

Benefits :

- Automatic image optimization
- Supports modern formats (WebP, AVIF)
- Responsive images
- Better Core Web Vitals

4 Environment Variables

- Store sensitive data like API keys, DB URLs etc.
- Use .env.local file in root directory.
- Access using `process.env.NEXT_PUBLIC_*` (for client) or `process.env.*` (for server only).

.env.local

```
NEXT_PUBLIC_API_URL=https://api.example.com
DB_URL=mongodb://localhost:27017/mydb
SECRET_KEY=your_secret_key
```

Only variables with `NEXT_PUBLIC_` are exposed to the browser. 😊

5 Error Handling

- Next.js provides a default error page and allows custom error handling.
- You can create `error.js` (App Router) or `_error.js` (Pages Router).

app/error.js (App Router)

```
"use client"
export default function Error({ error }) {
  return (
    <div>
      <h1>Something went wrong!</h1>
      <p>{error.message}</p>
    </div>
  )
}
```

pages/_error.js (Pages Router)

```
export default function Error({
  statusCode
}) {
  return (
    <div>
      <h1>Error {statusCode}</h1>
      <p>Something went wrong!</p>
    </div>
  )
}
```

6 Custom Document (Pages Router)

- Allows you to modify the `<html>` and `<body>` tags.
- Useful for adding meta tags, fonts, or third-party scripts.

pages/_document.js

```
export { Html, Head, Main, NextScript } from 'next/document'
export default function Document() {
  return (
    <Html lang="en">
      <Head>
        <meta name="description" content="My Next.js App" />
      </Head>
      <body>
        <Main />
        <NextScript />
      </body>
    </Html>
  )
}
```

Pro Tip: Use built-in features of Next.js instead of adding extra libraries.

Cleaner code.
Faster apps.
Happier users! 😊

Deployment & Optimization in Next.js

"Ship it,
Make it faster,
Repeat!" 😊

1 Deployment Options

You can deploy a Next.js app on multiple platforms. Most common options:

- **Vercel** (recommended) - made by Next.js creators.
- **Netlify** - easy and developer friendly.
- **AWS** (Amplify / EC2) - for custom setups.
- **Docker** - for containerized deployment.
- **Any VPS** - (DigitalOcean, Azure, GCP etc.).



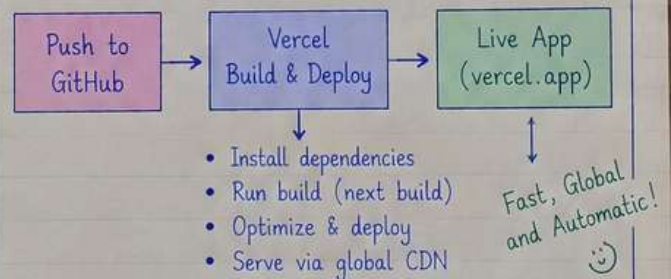
Why Vercel?

- Zero configuration deployment.
- Automatic CI/CD (from GitHub).
- Global CDN (fast worldwide).
- Preview deployments for every PR.
- Easy environment variable management.
- Built by the Next.js team. 😊

2 Deploy on Vercel (Step by Step)

- 1 Push your code to GitHub.
- 2 Go to vercel.com and import your repo.
- 3 Select the project and click Deploy.
- 4 Vercel automatically detects Next.js settings.
- 5 Your app will be live at <https://your-app.vercel.app>
- 6 Every new push to main branch will auto-deploy!

Deployment Flow (Vercel)



3 Environment Variables

- Used for sensitive data (API keys, DB URLs, secrets).
- In Vercel, go to **Project Settings** → **Environment Variables**.
- You can set different variables for:
 - Development
 - Preview (for PRs)
 - Production

.env.local (local)

```

NEXT_PUBLIC_API_URL=http://localhost:3000
SECRET_KEY=your_secret_key
  
```

Variables starting with `NEXT_PUBLIC_` are exposed to the browser. 😊

4 Build Command

Next.js uses the following commands:

```

# Install dependencies
npm install

# Build the application
npm run build

# Start the application (for custom server)
npm start
  
```

On Vercel, you don't need to run these manually. It's automated! 😊

5 Output

Next.js generates an optimized production build.

.next/ (build output)

```

├── server/ (server code)
├── static/ (static assets)
├── standalone/ (for self-hosting)
└── ...
  
```

This output is optimized for performance and smaller bundle size. 😊

6 Performance Optimization Tips

- Use **Image** component (automatic optimization).
- Prefer **Server Components** (less JS on client).
- Use dynamic imports for heavy components.
- Enable caching and revalidation (ISR).
- Analyze bundle size using `next build` and `@next/bundle-analyzer`.
- Use **CDN** (Vercel provides this automatically).
- Remove unnecessary dependencies.



Extra Tips:

- Use latest Next.js version.
- Keep an eye on build size.
- Use ESLint and TypeScript.
- Set up **monitoring** (e.g. Vercel Analytics, Sentry).
- Test on a preview deployment before production.
- Keep your app modular and clean. 😊

Next Page: → Bonus - Useful Tools, Resources & Summary

Deploy. Optimize. Grow. 😊

Next.js - Summary & Key Takeaways

"Consistent learning builds real skills."



1 What We Learned ?

- ✓ What is Next.js and why to use it
- ✓ Project structure and file-based routing
- ✓ Rendering (SSG, SSR, CSR, ISR)
- ✓ Data fetching methods
- ✓ API Routes (backend in Next.js)
- ✓ State management (local, context, server state)
- ✓ Static generation and dynamic routes
- ✓ Environment variables
- ✓ Deployment (Vercel and other platforms)
- ✓ Performance optimization techniques
- ✓ Advanced concepts and best practices

2 When to Use Next.js ?

- You need **SEO friendly** pages.
- Your application has both frontend and **backend logic** (API routes).
- You want **fast performance** and **optimized loading**.
- You are building a **modern web** application (e.g. blogs, dashboards, SaaS products).
- You want **easy deployment** and scalability.
- You prefer a **developer-friendly** experience with a large community.



Remember :

- Next.js is not just a React framework, it's a full-stack solution.
- Choose the right rendering strategy for your use case.
- Keep your app modular, clean and optimized.
- Deploy **early**, get feedback, and keep improving. 😊

3 Real-World Use Cases

- Blogs & News Websites (SSG)
- E-commerce Platforms (SSR / ISR)
- SaaS Dashboards (SSR + API Routes)
- Marketing / Landing Pages (SSG)
- Documentation Sites (SSG)
- Internal Tools & Admin Panels (SSR)
- Any modern web app that needs speed, SEO and scalability.

4 Pros and Cons

Pros 👍
✓ Fast performance
✓ SEO friendly
✓ Full-stack (frontend + backend)
✓ Flexible rendering options
✓ Large community
✓ Easy deployment (Vercel)
✓ Great developer experience

Cons 👎
✗ Slightly higher learning curve
✗ Can be overkill for small projects
✗ Some features are Vercel-centric
✗ Build size can grow if not optimized
✗ Requires understanding of modern React concepts

5 Learning Path (Next Steps)

- 1 Build small projects (blog, portfolio, todo app)
- 2 Explore **advanced features** (ISR, Middleware, i18n)
- 3 Integrate a **database** (e.g. MongoDB, PostgreSQL)
- 4 Add **authentication** (e.g. NextAuth.js)
- 5 Deploy on Vercel
- 6 Work on a real-world project
- 7 Keep exploring and stay updated !

Practice
Build
Repeat
Grow
😊

6 Useful Resources

- 📄 Official Docs : <https://nextjs.org/docs>
- 📺 YouTube : [Next.js Official Channel](#)
- 🌐 Vercel : <https://vercel.com>
- 🐙 GitHub : <https://github.com/vercel/next.js>
- 📄 Next.js Examples : <https://github.com/vercel/next.js/tree/canary/examples>
- 📖 Practice Projects : Build and deploy your own ideas 😊

7 Final Thoughts

- Next.js helps you build fast, modern and scalable web applications.
- It's widely used in the industry and constantly evolving.
- The best way to **learn** is by building and experimenting.
- Keep your fundamentals strong (React, JavaScript, Web concepts).
- Explore new features and contribute to open source if possible.
- Most importantly → keep learning !

"A **small** project today can become a **big opportunity** tomorrow." 😊

Happy Learning !

